

Igel Aergen - Adil Oubninte, Noé Vincent

Temps dévoué au projet :

- Adil Oubninte: env. x heures
- Noé Vincent: env. 30 heures

I. Introduction

Igel Aergen est un jeux de plateau se jouant à plusieurs (2 à 6 joueurs normalement), l'objectif ici à été de produire un programme C permettant de jouer à ce jeu au travers d'un seul et même terminal d'ordinateur ou, dans notre cas, en réseau.

Ce projet permet d'explorer différentes facettes et différents niveaux de la programmation C, allant du réseau à la gestion de structures abstraites. De plus, il nous aura permis d'utiliser et de nous familiariser encore un peu plus avec des outils permettant le bon développement d'un projet: la compilation automatisée avec CMake et la collaboration avec Git.

II. Première (petite) extension - Resizable Char Array

Objectif

Dans un souci d'économie de la mémoire et de résilience face à des cas extrêmes (comme des tests à 25 joueurs ayant 300 hérissons chacun, exemple arbitraire, ne pas sous-entendre que le programme y résiste) nous avons décidé d'implémenter les petites piles qui constituent les cases à l'aide de tableaux dynamique.

Réalisation

Un module `ResizableCharArray` a été créé qui permet le type `rca_t`. Celui-ci est utilisé pour stocker les hérissons situés sur une case. L'implémentation est classique, les tableaux sont initialisés de sorte à n'utiliser que peu de mémoire, l'espace alloué est doublé de taille à chaque dépassement de la capacité. Ceci permet d'éviter d'allouer, pour chaque case, l'espace nécessaire à stocker les hérissons dans le pire cas possible, c'est-à-dire tout les hérissons sur la même case.

III. Principale extension - Mode multijoueur en réseau

Objectif

Afin de permettre un mode multi-joueur plus confortable, pour les joueurs, nous avons décidé d'implémenter une solution permettant de jouer à plusieurs depuis différentes machines sur le même réseau local (ou sur des réseaux différents mais dont des ports spécifiques ont été ouverts).

Réalisation

Pour ce faire, une architecture client-serveur à été choisie.

Le client executera la fonction `client()` de `multi.c`, celle-ci prends comme argument une structure portant diverses informations de connection et paramètres de la partie.

Le serveur, lui, à un rôle un peu plus complexe. En effet, le serveur est aussi joueur, il executera donc en parallèle :

- la fonction `serveur()` (aussi située dans `multi.c`) permettant d'orchestrer les différentes phases du jeux et la synchronisation des plateaux entre les joueurs.
- la fonction `client()` connectée au serveur hébergé sur la même machine, permettant au propriétaire de la machine de jouer aussi.

La communication entre clients et serveur se fait au travers de sockets réseau, à l'aide du module `csapp.h`. Ainsi cette extension fait intervenir 2 composantes imprévues pour le projet: le gestion de sockets et le multithreading.

IV. Bibliographie

- Module `csapp.h`, Carnegie Mellon University, [lien vers csapp.c](#), [lien vers csapp.h](#)

Ce module a été utilisé selon les conseils de Guillaume Didier, enseignant le Réseau au sein du Module SYS en 1ère année du département d'informatique de l'ENS de Rennes. Ce module a permit de simplifier grandement la gestion des sockets réseau et de ce fait, a permit la communication entre clients et serveur.

- Cours de Guillaume Didier, SYS-L3 INFO ENS Rennes

Ce cours a été utile dans la compréhension et la mise en pratique du module `csapp` cité plus haut.

- StackOverFlow

Forum utilisé dans des cas marginaux pour traiter des bizarreries de C, en particulier pour le traitement des strings et buffers.